

Project Report

Name: Suhas-01 & Prakhar-09

Project link: [Github](#) & [Website](#)

Report Date: October 25, 2025

Summary

This document provides a comprehensive architectural analysis of the `SketchCalibur` project. The project's objective is to replicate the core functionality of the Excalidraw whiteboarding tool with additional functionalities focusing on multi-user, real-time collaboration.

`SketchCalibur` is a real-time collaborative whiteboard. It functions as a virtual canvas where multiple users can join a shared "room" and draw together simultaneously. The core functionality includes:

- **Real-Time Collaboration:** Users can see the drawings and cursor movements of other participants in the same room instantly.
- **Basic Drawing Tools:** Provides essential tools like a freehand pencil, a rectangle tool, and text entry.
- **Tool Options:** Allows users to select colors and other properties for their drawing tools.
- **Room-Based Sessions:** Collaboration is instanced into unique, shareable rooms.
- **Persistence:** Drawings are saved to a database, allowing users to rejoin a room and find their work preserved.

The application is architected as a full-stack MERN application. The architecture is logically divided into several primary domains:

1. **Frontend (Client):** A dynamic React single-page application (SPA) responsible for all UI and rendering.
 - **Technologies:** React, React Hooks (`useEffect`, `useContext`, `useReducer`), Tailwind CSS, Socket.io-Client, HTML5 Canvas.
2. **Backend (Server):** A Node.js/Express-based RESTful API that handles room management and persistence.
 - **Technologies:** Node.js, Express.js.
 - **Database:** MongoDB, Mongoose.
3. **Real-Time Layer:** A WebSocket (Socket.io) implementation that manages the bi-directional communication for synchronizing all collaborative events.
4. **DevOps & Deployment:** The complete pipeline for containerization and automated deployment.
 - **Technologies:** Docker/Docker Compose, GitHub Actions (CI/CD), Vercel (Frontend), Render (Backend).

I. Frontend Architecture: The Interactive Canvas

The frontend is the core of the user experience, responsible for capturing user input (drawing, tool selection) and rendering the collaborative canvas.

A. Responsive and Interactive UIs using Tailwind CSS

The project leverages Tailwind CSS to build a utility-first, responsive interface. This is essential for a tool-based application.

- **Implementation:** Tailwind is used to style the main toolbar (holding tools like pencil, rectangle, text), the options panels (color, stroke width), and the main canvas container.
- **Responsiveness:** Utility classes like `flex`, `grid`, and responsive prefixes (e.g., `md:flex-row`, `sm:w-full`) are employed to ensure the toolbar and canvas are usable on both large desktop monitors and touch-based mobile devices. The layout adapts to screen size, preventing UI elements from obscuring the drawing area.

B. Experimentation with React Hooks (`useEffect`, `useContext`, custom hooks)

React Hooks are fundamental to the frontend's logic and state management.

- **`useEffect`:** This hook is the primary driver for managing side effects. Its key roles are:
 1. **WebSocket Connection:** On component mount (e.g., in a `Room` component), `useEffect` initiates the WebSocket connection to the server.
 2. **Canvas Event Listeners:** It sets up and tears down DOM event listeners for the canvas (`mousedown`, `mousemove`, `mouseup`, `touchstart`, `touchmove`, `touchend`).
 3. **Socket Event Listeners:** It subscribes to WebSocket events (e.g., `draw-data-received`, `user-cursor-moved`) to receive updates from other users.
 4. **Cleanup:** The return function from `useEffect` is critical for cleaning up listeners and disconnecting the socket when the user leaves the room to prevent memory leaks.
- **`useContext`:** Provides global state to the entire component tree, eliminating "prop drilling." The primary context (e.g., `ToolContext`) would expose the currently selected tool, color, stroke style, and the functions to update them.
- **Custom Hooks:** The project's architecture would be significantly simplified by custom hooks. A `useDrawing` hook, for example, would encapsulate all the complex logic for handling canvas events, tracking the drawing state (`isDrawing`, `startPoint`), and emitting drawing data to the WebSocket server.

C. Complex State Management (Context API)

For an application of this type, the **Context API (likely combined with `useReducer`)** is a more lightweight and suitable choice than Redux.

- **State Managed:**

1. **Tool State:** The currently selected tool, color, and options (managed by a `ToolContext`).
 2. **Elements State:** The array of all drawn elements (lines, shapes, text) on the canvas. A `useReducer` hook is ideal here, with actions like `ADD_ELEMENT`, `UPDATE_ELEMENT`, and `SET_ELEMENTS` (for loading from the server).
- **Workflow:** When a user finishes drawing a shape (`mouseup`), the `useDrawing` hook dispatches an `ADD_ELEMENT` action to the local reducer *and* emits the new element data via the `WebSocket`. Other clients receive this data and dispatch the same `ADD_ELEMENT` action to their reducers, ensuring all canvases are synchronized.

II. Backend Architecture: The Central Hub

The backend serves as the persistent storage layer and the central coordinator for real-time communication.

A. REST API Design with MongoDB + Mongoose

The REST API is not responsible for the real-time drawing itself; it handles persistence and room lifecycle.

- **Endpoints:**
 - `POST /api/room/create`: Generates a unique room ID, creates a new `Room` document in MongoDB, and returns the room ID.
 - `GET /api/room/:roomId`: Fetched when a user first joins a room. This endpoint queries the database for the specified `roomId` and returns the existing array of drawing elements so the new user's canvas can be populated with the current drawing.
 - `PUT /api/room/:roomId/save`: A potential endpoint to explicitly save the current state of the canvas (though this is often handled automatically via `WebSockets`).
- **MongoDB + Mongoose Schema:** A `Room` schema is defined using Mongoose:
 - ```
const RoomSchema = new mongoose.Schema({
 roomId: { type: String, required: true, unique: true },
 elements: { type: String }, // Stored as a JSON string to be parsed
 createdAt: { type: Date, default: Date.now }
});
```

### B. Secure, Production-Ready RESTful APIs

Production readiness is achieved by:

- **Environment Variables:** Using `.env` files to manage sensitive data like the `DATABASE_URL`, `PORT`, and `JWT_SECRET`.
- **Error Handling:** Implementing a global error-handling middleware in Express to catch and format errors, preventing stack traces from leaking to the client.

- **Async/Await:** Using `async/await` in all controller functions that interact with the Mongoose models to ensure non-blocking database operations.

### C. Authentication and User Roles with JWT

While the base project allows anonymous access via room IDs, a JWT (JSON Web Token) system is the standard way to add user accounts and roles.

- **Implementation:**
  1. **Endpoints:** `POST /api/auth/register` and `POST /api/auth/login`.
  2. **Login:** On success, the server creates a JWT (signed with `JWT_SECRET`) containing the `userId` and `username`.
  3. **Client:** The client stores this JWT in `localStorage` or a secure cookie.
  4. **Role-Based Access:** When a JWT is present, the user could be designated the 'owner' of a room they create. A protected middleware on the backend (`verifyToken`) would check this token. An 'owner' might have exclusive rights, such as `DELETE /api/room/:roomId` (to delete the room) or a special WebSocket event to `clear-canvas`.

### D. Validating RESTful APIs using Postman

Postman is a development tool used to test the backend API *before* integrating it with the React frontend.

- **Process:** The developer would use Postman to send `POST` requests to `http://localhost:5000/api/room/create` to ensure a new room is created. They would then use a `GET` request to `http://localhost:5000/api/room/NEW_ROOM_ID` to verify that the correct room data (or an empty array) is returned. This validates the backend logic in isolation.

## III. Real-Time Communication Layer

### A. Real-Time Communication via WebSockets

This is the most critical component of the application. The project uses a WebSocket library, almost certainly **Socket.io**, for its robust room-based broadcasting features.

- **Architecture:** The Socket.io server is initialized and attached to the Express HTTP server.
- **Event-Driven Workflow:**
  1. **Connection:** A client connects and `socket.emit('join-room', { roomId })`.
  2. **Server (Join):** The server receives this and `socket.join(roomId)`. It then fetches the room's current data from MongoDB and `socket.emit('load-canvas', elements)` to the *new client only*.

3. **Client (Drawing):** When a user draws, `socket.emit('draw-data', { roomId, elementData })`.
4. **Server (Broadcast):** The server receives `draw-data` and broadcasts it to all *other* clients in the room: `socket.to(roomId).emit('draw-data-received', elementData)`.
5. **Client (Receiving):** Other clients listen for `draw-data-received` and update their local `elements` state, causing React to re-render the canvas with the new shape.
6. **Cursor Movement:** A separate, high-frequency event (`cursor-move`) is emitted and broadcast to show other users' cursors in real-time.

## IV. DevOps and Deployment Strategy

The project is configured for modern, automated development and deployment.

### A. Deploy Full-Stack Apps using Docker

The `docker-compose.yml` file is for **local development orchestration**. It defines and links the services required to run the entire application.

- **frontend service:** Builds the React app's Dockerfile and runs the development server (e.g., on port 3000).
- **backend service:** Builds the Node.js/Express server's Dockerfile and runs it (e.g., on port 5000).
- **mongo service:** Pulls the official `mongo` image from Docker Hub to run a database instance.
- **Benefit:** This allows any developer to clone the repo and run a single command (`docker-compose up`) to have the full stack running consistently.

### B. CI/CD Deployment with GitHub Actions + Render/Vercel

The `.github/workflows` directory enables Continuous Integration/Continuous Deployment.

- **Workflow Trigger:** The Action is triggered on a `push` to the `main` branch.
- **Jobs (Deployment):**
  1. **Deploy Frontend:** The workflow checks out the code, builds the static React application (`npm run build`), and pushes the resulting `build` folder to **Vercel**. Vercel is ideal for hosting static frontends.
  2. **Deploy Backend:** The workflow checks out the code (or builds a Docker image) and deploys the Node.js server to a platform like **Render**. Render can host backend services, databases, and Docker containers, making it a suitable choice for the MERN backend.

This CI/CD pipeline fully automates the process of moving code from a developer's `git push` to a live, updated production application.

## V. Outputs

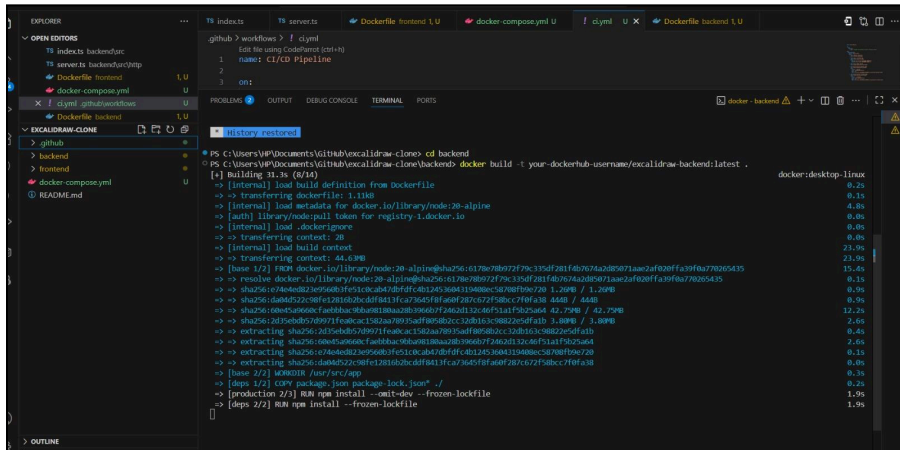


Figure 1: Docker build process for the backend service shown in the terminal.

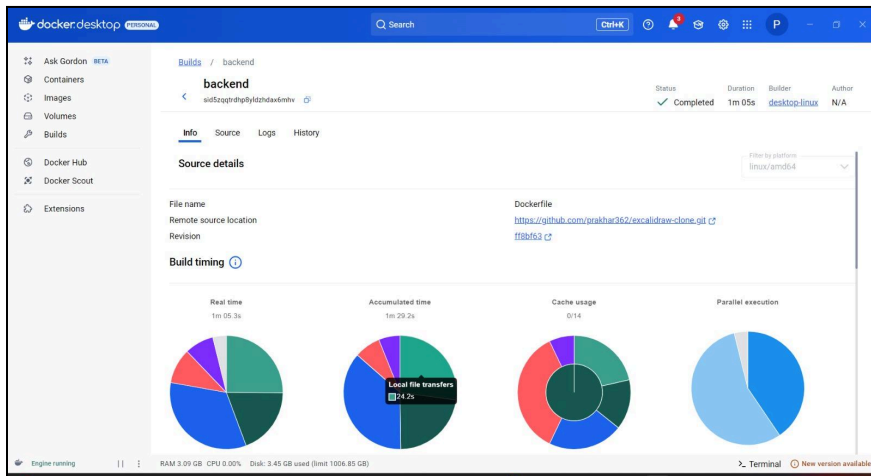


Figure 2: Docker Desktop view showing build details for the backend service.



Figure 3: Docker Desktop view showing build details for the frontend service.

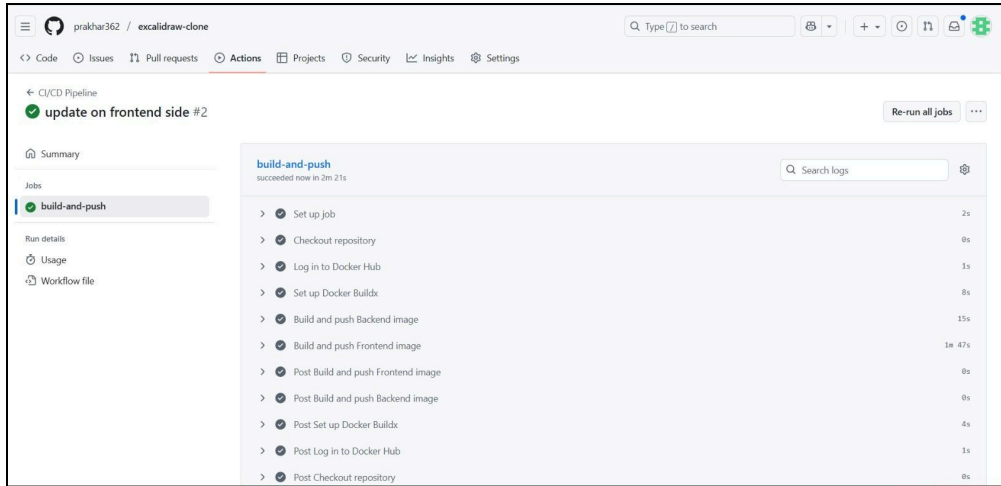


Figure 4: GitHub Actions CI/CD pipeline "build-and-push" job summary.

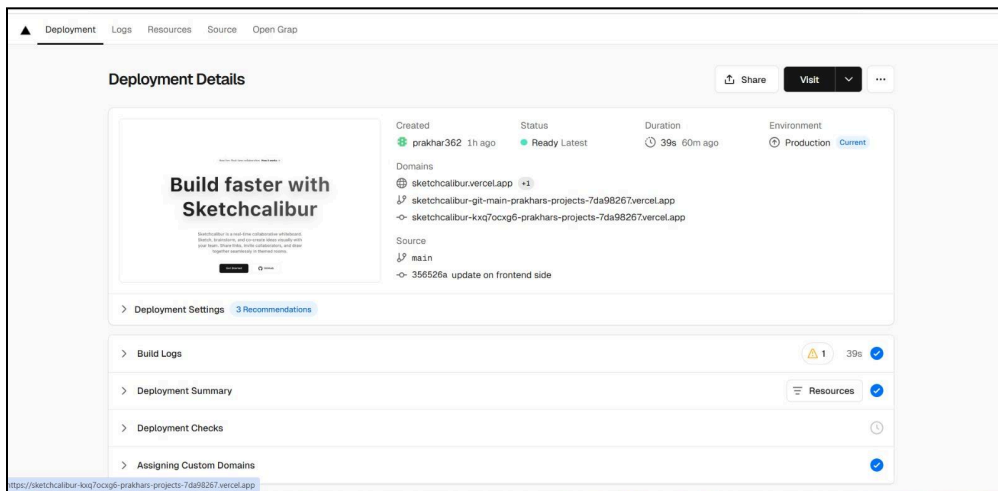


Figure 5: Vercel deployment details for the frontend, showing domains and "Ready" status.

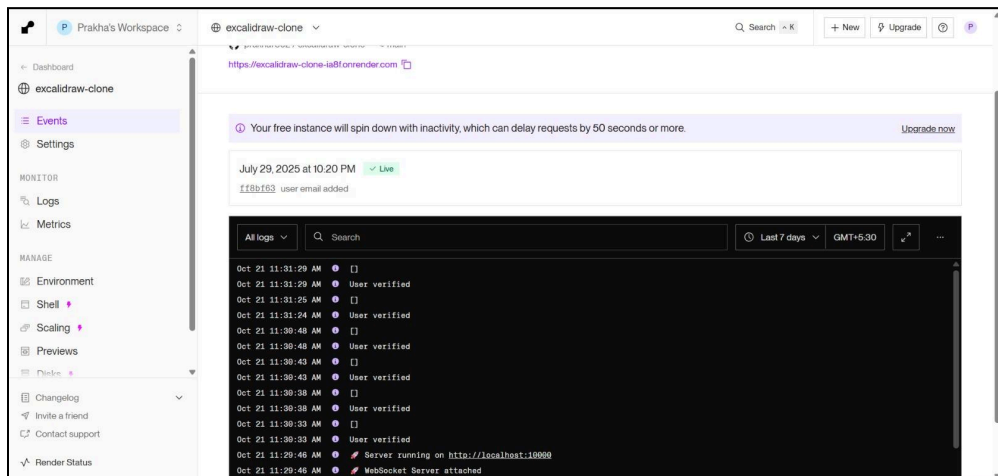


Figure 6: Render deployment logs for the backend, showing server running and WebSocket attachment.

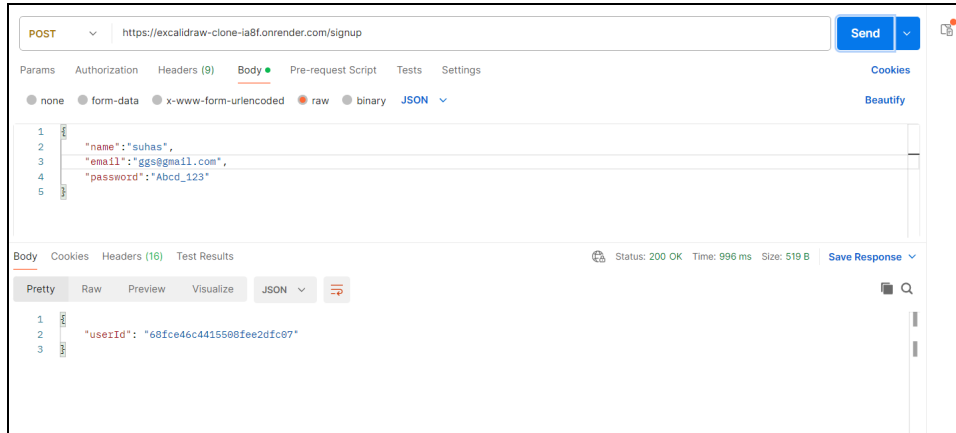


Figure 7: Postman API testing for the signup endpoint, showing a successful 200 OK response.

The image displays the authentication page of the SketchCalibur application. It features a 'Sign In' button and a 'Sign Up' button. Below the buttons is a 'Create Account' section with the instruction 'Make your new account here.' The form includes three input fields: 'Username' (placeholder: 'Choose your username'), 'Email' (placeholder: 'Enter your email'), and 'Password' (placeholder: 'Create a strong password' with a strength indicator icon). A 'Create Account' button is located at the bottom of the form.

Figure 8: Auth page of the SketchCalibur.

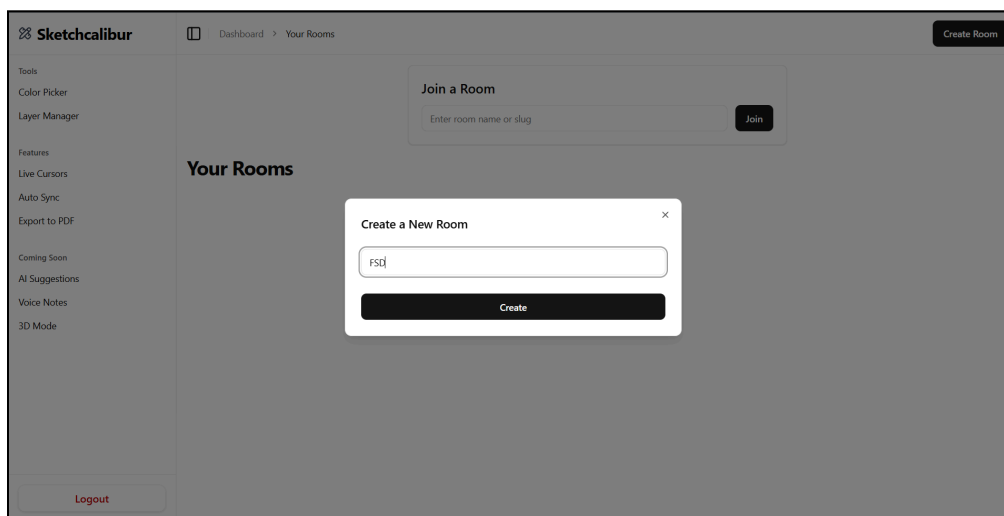


Figure 9: The user dashboard for "Your Rooms" to create a new room.



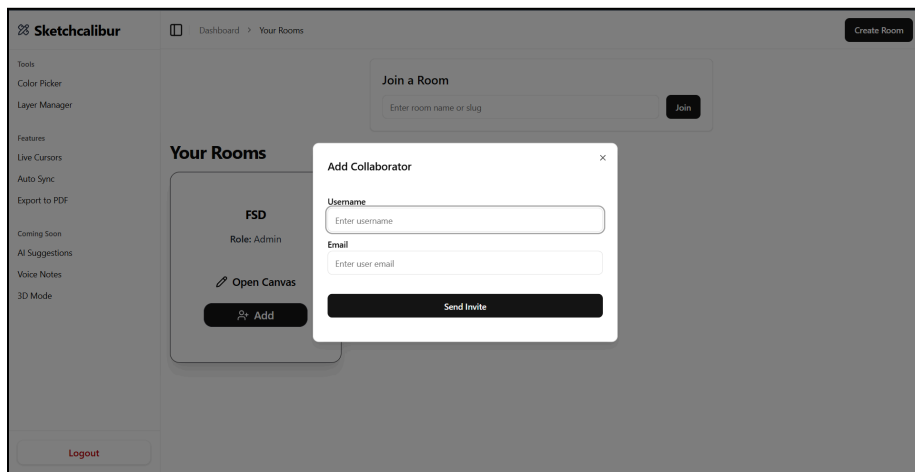


Figure 10: The user dashboard shows a room created and the modal to add a collaborator.

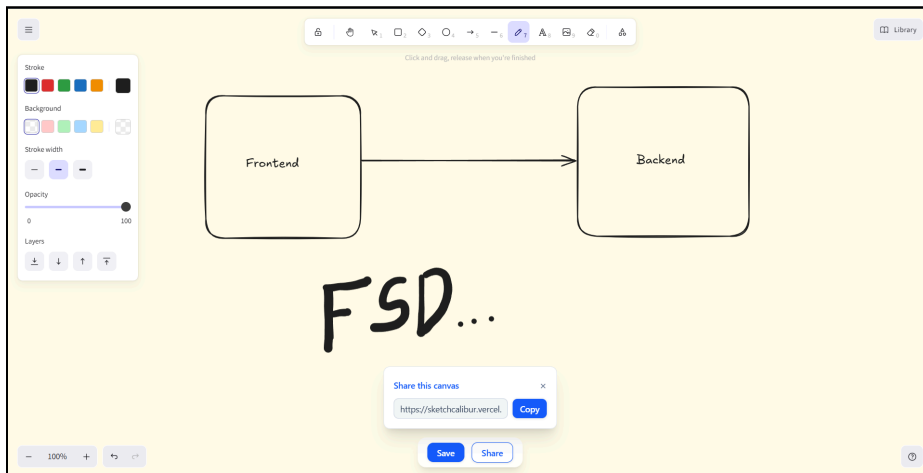


Figure 11: The SketchCalibur main whiteboarding canvas interface, showing tools and a drawing.

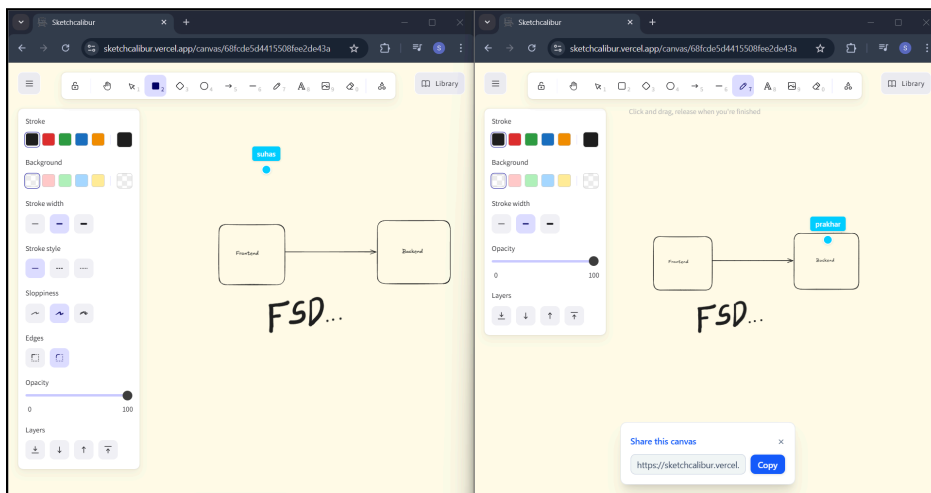


Figure 12: A side-by-side view demonstrating real-time collaboration with two users

## VI. Conclusion

The `excalidraw-clone` project serves as a comprehensive example of a modern, full-stack, real-time application. It successfully integrates a React-based frontend with a MERN-stack backend, using WebSockets as the critical communication layer to enable multi-user collaboration.

The architecture effectively separates concerns: React handles the view and user interaction, the Express API manages persistence and state, and Socket.io manages the real-time data synchronization. The inclusion of Docker and a CI/CD pipeline demonstrates a complete development lifecycle, from local orchestration to automated production deployment.